دستور زبان (Syntax) در پایتون

دستور زبان یا همان **سینتکس (Syntax)** یعنی «قوانین نگارش» یک زبان برنامه‌نویسی؛ درست مثل دستور زبان فارسی که می‌گوید فعل کجای جمله قرار می‌گیرد. اگر این قوانین رعایت نشوند، پایتون خطای نگارشی (**SyntaxError**) می‌دهد و اجرا نمی‌شود

۱. روش‌های اجرای کدهای پایتون

کدهای پایتون را می‌توان به دو روش اجرا کرد:

روش اول: حالت تعاملی (Interactive Mode / خط فرمان)

در ترمینال یا CMD دستور python را تایپ می‌کنید و علامت <<< ظاهر می‌شود. هر کدی که بنویسید، همان لحظه نتیجه را چاپ می‌کند:

```
>>> print("Hello, World!")
Hello, World!
```

- **کاربرد برای کلاس درس:** بسیار مناسب برای آزمون‌خطای سریع، تست توابع و یادگیری مفاهیم ابتدایی

روش دوم: ذخیره در قالب فایل اسکریپت :

کدها را داخل یک ویرایشگر می‌نویسیم و با پسوند py. ذخیره می‌کنیم (مثلاً app.py)، سپس با دستور زیر در ترمینال اجرا می‌کنیم:

```
python app.py
```

۲. تورفتگی (Indentation)؛ مهم‌ترین اصل در سینتکس پایتون

در زبان‌هایی مثل C یا Java از آکولاد {} برای مشخص کردن بلوک‌های کد استفاده می‌شود. اما در پایتون **تورفتگی (تعداد فاصله‌های ابتدای خط)** ساختار برنامه را تعریف می‌کند

قاعده اصلی

تمام دستوراتی که متعلق به یک شرط، حلقه یا تابع هستند، باید با فاصله مشخصی از ابتدای خط شروع شوند (استاندارد پایتون: **۴ عدد Space**).

```
if 5 > 2:
   print("پنج از دو بزرگ‌تر است!")
```

خطاهای متداول تورفتگی:

۱. **حذف تورفتگی (IndentationError):**

```
# این کد خطا می‌دهد:
if 5 > 2:
print("خطا رخ می‌دهد!")
```

۲. **ناهماهنگی در تعداد فاصله‌ها:**

تمام خطوط داخل یک بلوک باید دقیقاً به یک اندازه تورفتگی داشته باشند:

```
# خطای عدم تطابق فاصله:
if 5 > 2:
  print("خط اول")
    print("خط دوم خطا می‌دهد!")
```

۳. پایان دستورات و ادامه خط (Newlines)

- در پایتون، **رفتن به سطر بعدی (Enter)** به معنی پایان دستور است؛ نیازی به قرار دادن سمی‌کالن (;) در انتهای خط نیست
- اگر یک فرمول یا دستور طولانی بود و خواستید آن را در خط بعد ادامه دهید، از بک‌اسلش (\) یا پرانتز استفاده می‌شود:

```
# روش پرانتز (پیشنهادی و تمیزتر)
total = (10 + 20 +
    30 + 40)
```

۴. کامنت‌گذاری (Comments)

توضیحاتی هستند که پایتون آن‌ها را اجرا نمی‌کند و فقط برای خوانایی کد توسط انسان نوشته می‌شوند.

- **کامنت تک‌خطی:** با علامت #

```
# این خط سن کاربر را مشخص می‌کند
age = 16
```

- **توضیحات چندخطی (Docstring)**: با سه کوتیشن (""")

"""

این برنامه نمره دانش‌آموز را می‌گیرد

.و وضعیت قبولی یا رد شدن او را مشخص می‌کند

"""

۵. حساسیت به حروف کوچک و بزرگ (Case Sensitivity)

پایتون نسبت به بزرگی و کوچکی حروف بسیار حساس است. برای نمونه، سه متغیر زیر کاملاً با یکدیگر متفاوت‌اند:

```
name = "Ali"
Name = "Reza"
NAME = "Sara"
print(name)  # Ali را چاپ می‌کند
```

۶. کلمات کلیدی رزروشده (Keywords)

برخی کلمات دستورات اختصاصیِ خود پایتون هستند و **نمی‌توان** از آن‌ها به عنوان نام متغیر استفاده کرد؛ مانند:

if, else, while, for, def, class, import, True, False, None

```
for = 20  # این کار ممنوع است و ارور سینتکس می‌دهد:
```

۷. نام‌گذاری متغیرها (Variable Naming Rules)

قوانین ساخت نام متغیرها در سینتکس پایتون:

۱. باید با یک حرف انگلیسی یا خط زیر (_) شروع شوند

۲. نمی‌توانند با عدد شروع شوند (name۲ غلط است، اما name2 درست است)

۳. نباید شامل فاصله یا علامت‌های نگارشی (-, @, $) باشند

۴. روش رایج نام‌گذاری ترکیبی: سبک **snake_case** (مانند student_score)

تمرین :

اشکالات کدهای زیر را پیدا و تصحیح کنید :

#سوال ۱: چه خطایی در این کد وجود دارد؟

```
if 10 > 5:
print("بزرگ‌تر است")
```

```
سوال ۲: چرا کد زیر ارور می‌دهد؟ #
1st_score = 20
```

```
سوال ۳: خروجی این دو خط چیست؟ #
x = 5
print(X)
```

پاسخ‌ها :

۱. خطای تورفتگی (IndentationError) — قبل از print نیاز به چهار فاصله است

۲. خطای نام متغیر — متغیر نباید با عدد شروع شود

۳. خطای نام متغیر (NameError) — متغیر x تعریف شده، اما X فراخوانی شده است (حساسیت به حروف بزرگ/کوچک)

دستورها (Statements) در پایتون :

۱. دستور (Statement) چیست؟

یک برنامه کامپیوتری در واقع مجموعه‌ای از **«دستورالعمل‌ها»** (**Instructions**) است که کامپیوتر آن‌ها را یکی پس از دیگری **«اجرا»** (**Execute**) می‌کند

در زبان‌های برنامه‌نویسی، به این دستورالعمل‌ها **Statement** (به فارسی: **جمله** یا **دستور**) می‌گویند. هر Statement یک اقدام مشخص را به کامپیوتر ابلاغ می‌کند؛ مثلاً یک متغیر می‌سازد، محاسباتی انجام می‌دهد یا پیامی را روی صفحه نمایش چاپ می‌کند.

مثال ساده:

دستور زیر یک Statement مستقل است که متن "Python is fun!" را روی صفحه چاپ می‌کند:

```
print("Python is fun!")
```

۲. پایان دستور در پایتون (قاعده پایان خط)

در پایتون، بر خلاف بسیاری از زبان‌های دیگر (مانند C، C++، یا Java):

- برای پایان دادن به یک دستور **نیازی به سمی‌کالن (;) نیست**
- پایان هر سطر (زدن کلید **Enter**) به معنی پایان آن دستور است

۳. اجرای پشت‌سرهم دستورها (ترتیب اجرا)

بیشتر برنامه‌ها از چندین دستور تشکیل شده‌اند. کامپیوتر این دستورها را دقیقاً **به همان ترتیبی که نوشته شده‌اند، از بالا به پایین و خط به خط** اجرا می‌کند:

```
print("Hello World!")
print("Have a good day.")
print("Learning Python is fun!")
```

خروجی این برنامه در صفحه نمایش:

```
Hello World!
Have a good day.
Learning Python is fun!
```

نحوه تحلیل و اجرای برنامه توسط پایتون:

۱. ابتدا **دستور اول** اجرا می‌شود و جمله Hello World! را روی صفحه می‌آورد

۲. سپس مفسر به خط بعدی می‌رود و **دستور دوم** را اجرا می‌کند (Have a good day.)

۳. در نهایت، **دستور سوم** را اجرا کرده و پیام Learning Python is fun! را چاپ می‌کند

۴. استفاده از سمی‌کالن (;) — اختیاری و کم‌کاربرد

در پایتون نوشتن سمی‌کالن **اختیاری** است. اگر بخواهید چند دستور کوتاه را در یک خط بنویسید، می‌توانید بین آن‌ها از علامت ; استفاده کنید:

```
print("Hello"); print("How are you?"); print("Bye bye!")
```

خروجی:

```
Hello
How are you?
Bye bye!
```

نکته بسیار مهم :

با وجود این‌که پایتون به این کار اجازه می‌دهد، اما در برنامه‌نویسی واقعی و اصولی **به ندرت** از سمی‌کالن استفاده می‌شود؛ چرا که نوشتن چند دستور در یک خط، خواندن و پیدا کردن خطاهای کد را برای انسان دشوار می‌کند. فلسفه پایتون بر پایه **سادگی و خوانایی بالا** است؛ بنابراین همیشه توصیه می‌شود هر دستور را در یک خط جداگانه بنویسید

پرسش و تمرین کلاسی :

سوال ۱:

خروجی قطعه‌کد زیر به چه صورتی روی صفحه ظاهر می‌شود؟

```
print("شروع")
print("در حال پردازش...")
print("پایان")
```

سوال ۲:

آیا کد زیر از نظر نگارشی (Syntax) درست است؟ چه اشکال اصولی‌ای به آن وارد است؟

```
a = 10; b = 20; print(a + b)
```

- **پاسخ تشریحی برای مدرس:** کد کاملاً اجرا می‌شود و خروجی ۳۰ را می‌دهد؛ اما خوانایی پایینی دارد و بهتر است در ۳ سطر مجزا نوشته شود:

```
a = 10
b = 20
print(a + b)
```

خروجی و چاپ اطلاعات با دستور print() در پایتون

۱. تابع print چیست؟

تابع **print** یکی از پرکاربردترین و اولین دستوراتی است که در پایتون یاد می‌گیریم. وظیفه اصلی این تابع، **نمایش متن، اعداد و نتایج محاسبات روی صفحه نمایش (کنسول یا ترمینال)** است.

```
print("Hello World!")
```

خروجی:

```
Hello World!
```

۲. رفتار پیش‌فرض: رفتن خودکار به خط بعد

شما می‌توانید به هر تعداد که نیاز دارید از دستور print() استفاده کنید

نکته بسیار مهم: به‌طور پیش‌فرض، هر دستور print() پس از نمایش خروجی، نشانگر را به **خط بعدی (New Line)** می‌برد.

```
print("Hello World!")
print("I am learning Python.")
print("It is awesome!")
```

خروجی:

```
Hello World!
I am learning Python.
It is awesome!
```

۳. استفاده از نقل‌قول‌ها: دابل‌کوتیشن (") در برابر سینگل‌کوتیشن (')

در پایتون، به متن‌ها **رشته (String)** می‌گویند. تمام متن‌ها باید داخل علامت نقل‌قول قرار بگیرند. شما می‌توانید هم از **دابل‌کوتیشن** (") و هم از **سینگل‌کوتیشن** (') استفاده کنید و هیچ تفاوتی در عملکرد آن‌ها وجود ندارد:

```
print("This will work!")
print('This will also work!')
```

چه زمانی از کدام استفاده کنیم؟

اگر داخل متن خود نیاز به علامت نقل‌قول یا آپاستروف داشتید، می‌توانید از نوع دیگر در اطراف کل متن استفاده کنید تا ارور سینتکس نگیرید:

#استفاده از دابل‌کوتیشن برای متنی که خودش ' دارد:

```
print("It's a beautiful day!")
```

#استفاده از سینگل‌کوتیشن برای متنی که خودش " دارد:

```
print('گیدو ون روسوم گفت: "پایتون ساده است".')
```

۴. چاپ در همان خط با پارامتر end

همان‌طور که گفتیم، پایتون در انتهای هر print به‌طور مخفیانه کاراکتر رفتن به سطر بعد (/n) را اضافه می‌کند

اگر بخواهید **چند دستور print متوالی در یک خط چاپ شوند**، باید از شناسه یا آرگومان **end** استفاده کنید:

```
print("Hello World!", end=" ")
print("I will print on the same line.")
```

خروجی:

```
Hello World! I will print on the same line.
```

توضیح نحوه کار end:

- در مثال بالا، دستور داده‌ایم که انتهای خط اول به جای رفتن به سطر بعد، با یک **فاصله** (**Space**) تمام شود
- حتی می‌توانید علامت‌های دیگر در end بگذارید:

```
print("فصل اول", end=" -> ")
print("فصل دوم", end=" -> ")
print("پایان")
```

خروجی:

```
فصل اول -> فصل دوم -> پایان
```

۵. قابلیت‌های تکمیلی و کاربردی

الف) چاپ چندین مقدار کنار هم با کاما (,)

با گذاشتن کاما بین مقادیر داخل print، پایتون به طور خودکار بین آن‌ها یک فاصله می‌گذارد:

```
name = "Ali"
score = 20
print("نام دانش‌آموز:", name, "نمره:", score)
```

خروجی:

نام دانش‌آموز: Ali نمره: ۲۰

ب) جداکننده دلخواه با پارامتر sep

به‌طور پیش‌فرض کاما یک فاصله بین عبارات می‌اندازد. با استفاده از sep می‌توانید تعیین کنید چه علامتی بین آن‌ها قرار بگیرد:

```
print("روز", "ماه", "سال", sep="/")
```

خروجی:

روز/ماه/سال

ج) چاپ اعداد و نتایج محاسبات

برای چاپ اعداد نیازی به گذاشتن کوتیشن نیست. اگر عدد را داخل کوتیشن نگذارید، پایتون آن را محاسبه می‌کند:

```
print(10 + 5)   # خروجی: ۱۵ (محاسبه ریاضی)
print("10 + 5")  # خروجی: ۱۰ + ۵ (متن ساده)
```

۶. جدول جمع‌بندی پارامترهای print()

content_copy

ویژگی / پارامتر	وظیفه	مقدار پیش‌فرض
end	تعیین کاراکتری که در انتهای دستور چاپ می‌شود	/n (رفتن به خط بعد)
sep	تعیین کاراکتر جداکننده بین چند آیتمِ ورودی	" " (یک فاصله)

۷. تمرین و فعالیت کلاسی

تمرین ۱: کد زیر چه خروجی‌ای تولید می‌کند؟

```
print("یک", end="-")
print("دو", end="-")
print("سه")
```

- **پاسخ:** یک-دو-سه

تمرین ۲: از دانش‌آموزان بخواهید با استفاده از print() و پارامتر end، یک شمارش معکوس به شکل زیر روی یک سطر بنویسند:

... 1 ... 2 ... 3پرتاب!

کد پاسخ #:

```
print("3", end=" ... ")
print("2", end=" ... ")
print("1", end=" ... ")
print("پرتاب!")
```

چاپ اعداد و انجام محاسبات در دستور printپایتون :

۱. تفاوت مهم: متن در برابر عدد

یکی از اولین نکاتی که هر برنامه‌نویس باید بداند، تفاوت میان **متن** (**String**) و **عدد** (**Number**) است:

- **متن‌ها** حتماً باید داخل نقل‌قول (کوتیشن ' ' یا دابل‌کوتیشن " ") باشند
- **اعداد** نیازی به کوتیشن **ندارند** و مستقیماً نوشته می‌شوند

```
print(3)
print(358)
print(50000)
```

خروجی:

3

358

50000

نکته کلیدی:

اگر یک عدد را داخل کوتیشن بگذارید، پایتون با آن مثل یک کلمه معمولی رفتار می‌کند، نه یک عدد قابل محاسبه!

- ۵ یک **عدد** است
- "۵" یک **متن (کاراکتر متنی)** است

۲. انجام محاسبات ریاضی داخل print

پایتون می‌تواند مانند یک ماشین‌حساب هوشمند عمل کند. اگر عملگرهای ریاضی را بین اعداد بنویسید، پایتون ابتدا حاصل را حساب کرده و سپس جواب نهایی را چاپ می‌کند:

```
print(3 + 3)
print(2 * 5)
```

خروجی:

6

10

عملگرهای ریاضی اصلی در پایتون:

عملیات	عملگر در پایتون	مثال در کد	نتیجه
جمع	+	print(۱۲ + ۸)	۲۰
تفریق	-	print(۲۰ - ۷)	۱۳
ضرب	*	print(۴ * ۶)	۲۴
تقسیم اعشاری	/	print(۱۰ / ۲)	۵.۰
تقسیم جزء صحیح	//	print(۱۱ // ۲)	۵
باقیمانده تقسیم	٪	print(۱۰ ٪ ۳)	۱
توان (به توان رساندن)	**	print(۲ ** ۳)	۸

یک مقایسه جالب برای درک بهتر:

print(5 + 5) # خروجی: ۱۰ (چون عدد هستند و جمع ریاضی انجام می‌شود)

print("5 + 5") # خروجی: ۵ + ۵ (چون داخل کوتیشن است و یک متن ساده است)

print("5" + "5") # خروجی: ۵۵ (دو متن را کنار هم می‌چسباند!)

۳. ترکیب متن و عدد در یک خروجی (Mix Text and Numbers)

در بیشتر برنامه‌ها نیاز داریم در کنار عدد، متنی هم برای توضیح آن بیاوریم (مثلاً نمایش سن، قیمت یا نمره)

ساده‌ترین و اصولی‌ترین روش، **استفاده از کاما (,)** بین متن و عدد است. پایتون به‌طور خودکار یک فاصله بین آن‌ها قرار می‌دهد:

```
print("I am", 35, "years old.")
```

خروجی:

```
I am 35 years old.
```

مثال کاربردی دیگر:

```
score = 19
print("نمره شما در آزمون:", score)
```

خروجی:

نمره شما در آزمون: ۱۹

۴. خطای رایج مبتدی‌ها: ترکیب متن و عدد با +

```
print("سن من: " + 15)  # این کد خطا می‌دهد (TypeError):
```

چرا خطا می‌دهد؟

علامت + در پایتون یا باید دو عدد را جمع کند، یا دو متن را به هم بچسباند. پایتون نمی‌تواند یک متن و یک عدد را با + مستقیماً ترکیب کند

دو راه‌حل ساده:

۱. **استفاده از کاما (روش پیشنهادی و راحت):**

```
print("سن من:", 15)
```

۲. **تبدیل عدد به متن با دستور str():**

```
print("سن من: " + str(15))
```

۵. مثال‌های ترکیبی و کاربردی برای تدریس در کلاس

مثال ۱: فاکتور خرید ساده

```
price = 25000
count = 4
print("قیمت هر کالا:", price, "تومان")
print("تعداد:", count)
print("مبلغ کل خرید:", price * count, "تومان")
```

خروجی:

قیمت هر کالا: ۲۵۰۰۰ تومان

تعداد: ۴

مبلغ کل خرید: ۱۰۰۰۰۰ تومان

مثال ۲: محاسبه مساحت و محیط مستطیل

```
tool = 8
arz = 5
print("محیط مستطیل:", (tool + arz) * 2)
print("مساحت مستطیل:", tool * arz)
```

خروجی:

محیط مستطیل: ۲۶

مساحت مستطیل: ۴۰

۶. پرسش‌ها و چالش کلاسی

سوال ۱: خروجی دستورهای زیر را حدس بزنید:

```
print(2 + 3 * 4)
```

```
print((2 + 3) * 4)
```

- **پاسخ:** در اولی اولویت با ضرب است (۱۴)؛ در دومی اولویت با پرانتز است (۲۰)
- **سوال ۲:** چرا اجرای خط زیر با ارور مواجه می‌شود و چطور باید آن را درست کنیم؟

```
print("نفر دانش‌آموز دارد " + 25 + " کلاس ما.")
```

- **پاسخ:** از علامت + بین متن و عدد استفاده شده است. باید به‌جای + از کاما (,) استفاده شود:

```
print("کلاس ما", 25, "نفر دانش‌آموز دارد.")
```

یادداشت‌ها و توضیحات (Comments) در پایتون

۱. کامنت (Comment) چیست و چرا استفاده می‌شود؟

کامنت‌ها یا **یادداشت‌های توضیحی**، بخش‌هایی از کد هستند که **توسط پایتون نادیده گرفته می‌شوند**؛ یعنی مفسر پایتون آن‌ها را اجرا نمی‌کند و هیچ تاثیری در خروجی برنامه ندارند

۳ کاربرد اصلی کامنت‌ها در برنامه‌نویسی:

۱. **توضیح دادن منطق کد:** برای اینکه خودتان یا دیگران بعداً متوجه شوید هر بخش از کد چه کاری انجام می‌دهد

۲. **افزایش خوانایی پروژه:** تقسیم‌بندی بخش‌های مختلف کد در پروژه‌های طولانی

۳. **غیرفعال کردن موقت کدها در زمان تست (عیب‌یابی / Debugging):** اگر بخشی از برنامه خطا داد یا نخواستید موقتاً اجرا شود، به جای پاک کردن، آن را کامنت می‌کنید

۲. روش ساخت کامنت تک‌خطی (Single-line Comment)

در پایتون، برای نوشتن کامنت از علامت هش یا مربع (**#**) استفاده می‌شود. هر متنی که بعد از علامت # بیاید، توسط پایتون خوانده نمی‌شود:

```
# این یک کامنت است و اجرا نمی‌شود
print("Hello, World!")
```

نوشتن کامنت در انتهای سطر کد (Inline Comment)

می‌توانید کامنت را دقیقاً روبروی یک خط کد قرار دهید. پایتون ابتدا دستور قبل از # را اجرا می‌کند و باقی خط را نادیده می‌گیرد:

print("Hello, World!") # چاپ پیام خوش‌آمدگویی

۳. غیرفعال کردن کدها برای تست و خطاگیری

گاهی اوقات برنامه‌نویسان نمی‌خواهند یک خط از برنامه اجرا شود، اما نمی‌خواهند آن را حذف کنند تا در آینده دوباره استفاده کنند. در این حالت، علامت # را ابتدای آن خط می‌گذارند:

```
# print("Hello, World!")
print("Cheers, Mate!")
```

خروجی:

```
Cheers, Mate!
```

(خط اول اجرا نشد چون تبدیل به کامنت شده است)

۴. نوشتن کامنت‌های چندخطی (Multiline Comments)

در پایتون بر خلاف زبان‌هایی مثل C یا Java (که دستور اختصاصی /* ... */ دارند)، سینتکس رسمی مجزایی برای کامنت چندخطی وجود ندارد. برای نوشتن توضیحات طولانی از دو روش استفاده می‌شود:

روش اول: قرار دادن # در ابتدای هر سطر (روش استاندارد)

```
# این یک کامنت توضیحی است
# که برای توضیحات بیشتر
# در چند سطر نوشته شده است
print("Hello, World!")
```

روش دوم: استفاده از متن چندخطی با سه‌کوتیشن (""" یا ''')

اگر یک متن چندخطی با سه علامت نقل‌قول باز و بسته شود، اما آن را داخل هیچ متغیری ذخیره نکنیم (انتساب ندهیم)، پایتون آن را می‌خواند اما بدون هیچ اثری از روی آن عبور می‌کند:

```
"""
این بخش از برنامه
مسئول محاسبه نمرات نهایی
.و رتبه‌بندی دانش‌آموزان است
```

"""

print("Hello, World!")

نکته تخصصی برای دبیران: به این سه‌کوتیشن‌ها در ابتدای توابع و کلاس‌ها **Docstring** (مستندات کد) نیز گفته می‌شود که در ساخت راهنماهای نرم‌افزاری کاربرد دارند

۵. کلید میان‌بر جادویی در ویرایشگرها

برای اینکه مجبور نباشید برای هر خط جداگانه # بگذارید:

- خطوط مورد نظر را با ماوس انتخاب کنید
- کلید ترکیبی **Ctrl** + / (در ویندوز) یا **Cmd** + / (در مک) را بزنید تا تمام خطوط انتخاب‌شده یک‌جا کامنت یا از حالت کامنت خارج شوند

۶. اصول و اخلاق کامنت‌نویسی تمیز (Best Practices)

۱. **واضحات را کامنت نکنند:** ❌ بد: x = 5 # متغیر x را برابر ۵ قرار بده ✅ خوب: score = 100 # بالاترین نمره آزمون نیم‌سال اول

۲. کامنت باید علتِ نوشتن یک کد (چرایی) را بگوید، نه صرفاً بازگو کردن خود دستور پایتون

۳. در کدهای تیمی، کامنت‌ها معمولاً انگلیسی یا به زبان مورد توافق تیم نوشته می‌شوند

۷. تمرین و آزمون کلاسی

سوال ۱: خروجی برنامه زیر چیست؟

```
# a = 10
b = 20
# b = b + 5
print(b)
```

- **پاسخ:** عدد ۲۰ (چون خطوط ۱ و ۳ کامنت هستند و اجرا نمی‌شوند)

سوال ۲: در کد زیر چه اشکالی وجود دارد؟

```
print("نمره دانش‌آموز:") # print(20)
```

- **پاسخ:** فقط عبارت نمره دانش‌آموز: چاپ می‌شود؛ چون دستور دوم بعد از علامت # آمده و کامنت شده است